

## String

String str = "Babbar";

length  $\Rightarrow$  str.length();

str.push-back('c');  $\Rightarrow$  Babbarc

str.pop-back();  $\Rightarrow$  Babbar

H.O.

- ① Check string & Read on website
- ② Key differences b/w char array & string

H.O.

Q- Reverse words of string, i/p  $\rightarrow$  My name is love  
o/p  $\rightarrow$  My name si evol

Q- Return maximum occurring character in a string

char getMaxOccCharacter (String s) {

int arr[26] = {0};

for (int i=0; i<s.length(); i++) {

char ch = s[i];

int number = 0;

if (ch >= 'a' && ch <= 'z') {

number = ch - 'a';

}

else {

number = ch - 'A';

}

arr[number]++;

int maxi = -1, ans = 0;

for (int i=0; i<26; i++)

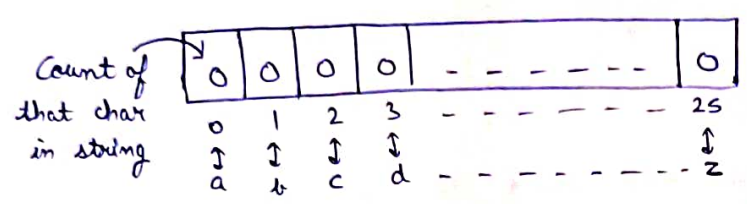
```

    if (maxi < arr[i]) {
        ans = i;
        maxi = arr[i];
    }
}

char finalAns = 'a' + ans;

return finalAns;
}

```



Now, agar koi char aa raha hai to uski place par count ++ Kar do  
 And fir end me pure array ka maximum nikal lo aur uske index ko  
 uske char se map Kar do.

⇒ For taking full input ⇒ cin.getline(str, len);

### H.W Custom delimiter

→ length

```
int len = strlen(name);
```

→ Compare (s1, s2)

```
strcmp(s1, s2)
```

→ Copy

```
strcpy(destination string, source string)
```

Q- You have been given a string str of words. You need to replace all the spaces b/w words with @40. I/P ⇒ My name is Khan  
 O/P ⇒ My@40name@40is@40Khan

### String replace spaces (string & str)

```

string temp = "";
for (int i=0; i<str.length(); i++) {

```

$$f([a]_{\sim} \times [a]_{\sim}) \subseteq \bar{f}$$
$$1.1 = 1.05$$
$$[1] \text{ mod } 10 = 1$$

1

}

```
else {
```

temp.push\_back(str[i]);

3

3

```
return temp;
```

3

[illegible]

for trees  
work took  
months and

H.W. abcd ++ dnoo roq real q iden at ion odin so rudo ind s.c.  $\Rightarrow O(1)$  me  
Do previous space waste question without creating temp  
ex xabmi iden ruo ol laoter munimom ex jared xrooq enr bms rff bmt  
Kardes dikhaas

Lecture 1910

Ques- Remove all occurrences of a substring, i/p  $\Rightarrow$  daabcbqa bcbc, part  $\Rightarrow$  abc

s & part consists of lower case characters  $\Rightarrow$   $\downarrow$  data structure  $\Rightarrow$

↓  
d a b a b c estremities metes (a.H)

↓  
o/p ⇒ data ← signal  
normal mel x ta = mel tri

string removeOccurrences (string s, string part) {

```
while (s.length() != 0 && s.find(part) < s.length()) {
```

s.erase(s.find(part), part.length());

it the user } it has not. throw for its prints a msg and not -un-  
 mark is error if  $\leq 912$  . or @ then throw and msg  
 return s;  
 mark @ is or @ error if  $\leq 912$

3

LeetCode 567

Leetcode 567  
Ques - Permutation in string, Given two strings  $s_1$  and  $s_2$ , return true

if  $S_2$  contains a permutation of  $S_1$ , or false otherwise.



```
bool checkInclusion(string s1, string s2) {
```

```
    int count1[26] = {0};
```

```
    for (int i=0; i<s1.length(); i++) {
```

```
        int index = s1[i] - 'a';
```

```
        count1[index]++;
```

```
    }
```

```
    // Traverse s2 string in window of size s1 length and compare
```

```
    int i=0;
```

```
    int windowSize = s1.length();
```

```
    int count2[26] = {0};
```

```
    // Running for first window
```

```
    while (i < windowSize) {
```

```
        int index = s2[i] - 'a';
```

```
        count2[index]++;
```

```
        i++;
```

```
    }
```

```
    if (checkEqual(count1, count2)) {
```

```
        return 1;
```

```
    }
```

```
    // aage window process karo
```

```
    while (i < s2.length()) {
```

```
        char newChar = s2[i];
```

```
        int index = newChar - 'a';
```

```
        count2[index]++;
```

```
        char oldChar = s2[i - windowSize];
```

```
        int oldIndex = oldChar - 'a';
```

```
        count2[oldIndex]--;
```

```
        i++;
```

```
        if (checkEqual(count1, count2))
```

```
            return 1;
```

Code for checkEqual

```
bool checkEqual(int a[26], int b[26])
```

```
{
    for (int i=0; i<26; i++) {
```

```
        if (a[i] != b[i])
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

leetcode 1047

H.W. Qu- Remove all adjacent duplicates in string

i/p  $\Rightarrow$  a bb a c a

$\Downarrow$

a a c a

$\Downarrow$

o/p  $\Rightarrow$  ca

i/p  $\Rightarrow$  a z xx z o y = [as] t t r u s s t r i

$\Downarrow$

a zz y

$\Downarrow$

o/p  $\Rightarrow$  ay

leetcode 443

Qu- String Compression, i/p  $\Rightarrow$  {a, a, b, b, b, c, c, c}

o/p  $\Rightarrow$  {a, 2, b, 3, c, 3}

exception  $\rightarrow$  if count of char is 1, then only show char

i/p  $\Rightarrow$  {a, a, b, c, c, d, d}

o/p  $\Rightarrow$  {a, 2, b, c, 2, d, 2}

int compress(vector<char> & chars) {

int i = 0;

int ansIndex = 0;

int n = chars.size();

while (i < n) {

int j = i + 1;

while (j < n && chars[i] == chars[j]) {

j++;

}

chars[ansIndex++] = chars[i];

int count = j - i;

if (count > 1) {

string cnt = to\_string(count);

for (char ch : cnt) {

chars[ansIndex++] = ch;

}

}

i = j;

}

return ansIndex;